

Unity 기반 골프 물리 시뮬레이션

기술 보고서

항목	내용
과목	컴퓨터그래픽스
프로젝트	기말 프로젝트 — Unity 기반 골프 물리 시뮬레이션
참고 게임	팡야 (Pangya)
팀 구성	개인 프로젝트
학번	202421246
이름	차가원
개발 플랫폼	Unity 6000.3.10f1 (Unity 6.3 LTS)
작성일	2026 년 6 월 17 일

1. 프로젝트 개요

본 프로젝트는 Unity 를 이용하여 골프 게임의 핵심 물리 현상을 Unity 내장 물리 엔진의 파라미터 조작이 아닌 C# 스크립트 내 수식 연산으로 직접 구현하는 것을 목표로 하였다. Rigidbody 와 Collider 는 기본적인 충돌 감지와 기본 중력 적용 용도로만 사용하였으며, 공의 궤적과 속도에 결정적인 영향을 미치는 바람 외력, 표면별 마찰 감속, 반발계수는 모두 스크립트 내에서 벡터 연산을 통해 명시적으로 계산하였다.

사용자는 Space 키를 눌러 발사 파워를 충전하고 A·D 키로 조준 피벗(aimPivot)을 회전시켜 발사 방향을 설정한다. Space 키를 때는 순간 공이 발사되며, 발사된 공은 중력과 바람의 영향을 받아 포물선 운동을 한다. 이동 중인 공에는 표면(페어웨이·러프·벙커)에 따라 다른 마찰이 적용되고, 벽 및 지면과의 충돌 시 반발계수를 이용한 반사 또는 바운스가 발생한다. 발사 전에는 바람을 반영한 예측 궤적선이 화면에 표시되며, 발사 시점과 정지 시점에 맞추어 조준 카메라와 추적 카메라가 자동으로 전환된다.

1-1. 평가 항목별 구현 위치

구현 항목	배점	구현 위치	핵심 수식 / 방식
발사체 운동	15 점	GolfBallController.Shoot(), Update()	$P = P_0 + Vt + \frac{1}{2}at^2$
외력 — 바람	10 점	WindManager.cs / FixedUpdate()	$F_{total} = F_{shot} + F_{wind}$
표면별 마찰	10 점	OnCollisionStay(), FixedUpdate()	$V_{new} = V_{old} \times \mu$
충돌 & 반발	15 점	OnCollisionEnter()	$V_{after} = e \times V_{before}$
궤적 예측	10 점	DrawTrajectory()	$P = P_0 + Vt + \frac{1}{2}at^2$ (+ 풍속항)
카메라 이동	10 점	CameraFollow.cs / Shoot(), FixedUpdate()	Lerp / Slerp 보간

2. 발사체 운동 (Projectile Motion)

사용 수식

$$P = P_0 + Vt + \frac{1}{2}at^2$$

P_0 : 공의 초기 위치

V : 초기 속도 벡터

a : 중력 가속도 (Unity 기본값, Physics.gravity $\approx$ (0, -9.81, 0) m/s²)

t : 경과 시간

구현 방법

사용자가 Space 키를 누르고 있는 동안 Update()에서 currentPower 를 chargeSpeed × Time.deltaTime 만큼 매 프레임 누적시키고, 0 ~ maxPower 범위로 Clamp 한다. Space 키를 때는 순간 Shoot()이 호출되어 수평 방향 성분(shootDirection × currentPower)과 수직 방향 성분

($\text{Vector3.up} \times \text{upwardPower}$)을 합산한 초기 속도(finalDirection)를 ForceMode.Impulse 로 Rigidbody 에 가한다. 이후 시간에 따른 위치 변화, 즉 위 수식의 적분은 Unity Rigidbody 의 기본 중력 기능을 통해 자동으로 계산된다 — 초기 속도(V) 산출은 직접 수식으로 구현하고, 위치의 시간 적분은 가이드라인에서 허용한 Rigidbody 의 기본 중력을 사용한 것이다.

```
// Update() - 파워 충전
if (Input.GetKey(KeyCode.Space))
{
    currentPower += chargeSpeed * Time.deltaTime;
    currentPower = Mathf.Clamp(currentPower, 0, maxPower);
}

// Shoot() - 초기 속도 생성 및 발사
Vector3 finalDirection =
    shootDirection * currentPower
    + Vector3.up * upwardPower;
rb.AddForce(finalDirection, ForceMode.Impulse);
```

파라미터 설정 근거

파라미터	값	설정 근거
maxPower	25	값이 클수록 코스를 쉽게 벗어나고, 작을수록 홀까지 도달하기 어려웠다. 반복 테스트 결과 25가 코스 규모에 맞는 적절한 발사 거리와 조작 난이도를 제공하였다.
upwardPower	3	공이 충분히 공중으로 떠오르면서도 지나치게 높이 솟아올라 체공 시간이 길어지지 않도록 3으로 설정하였다.
chargeSpeed	20	Space 키를 짧게 눌러도 약한 샷을, 길게 눌러도(약 1.25초 이상) 최대 파워에 도달하도록 충전 속도를 조정하였다.

3. 외력 — 바람 (Wind Force)

사용 수식

$$F_{total} = F_{shot} + F_{wind}$$

구현 방법

씬이 시작될 때 $\text{WindManager.Start()}$ 에서 랜덤 바람 벡터를 한 번 생성하며, 이후 라운드가 진행되는 동안 값은 고정된다. 공이 이동 중일 때 ($\text{rb.linearVelocity.magnitude} > 0.1$) $\text{GolfBallController.FixedUpdate()}$ 에서 매 프레임 이 바람 벡터를 외력으로 추가하여 발사 힘과 합성한다. 화면 우측 상단의 WindUI 는 x·z 성분 중 절댓값이 더 큰 축을 기준으로 방향 화살표를, 벡터의 크기를 세기 수치로 표시하여 사용자가 현재 바람 환경을 확인할 수 있도록 하였다.

```
// WindManager.Start() - 바람 벡터 생성(1회)
windForce = new Vector3(
    Random.Range(-2f, 2f), 0, Random.Range(-2f, 2f));

// GolfBallController.FixedUpdate() - 바람 적용
if (windManager != null && rb.linearVelocity.magnitude > 0.1f)
```

```
rb.AddForce(windManager.windForce);
```

파라미터 설정 근거

파라미터	값	설정 근거
Wind X	-2 ~ 2	너무 작은 범위는 궤적에 거의 영향을 주지 못했고, 너무 큰 범위는 조작이 거의 불가능할 정도로 궤적을 왜곡하였다. 반복 테스트 결과 ± 2 범위가 체감 가능하면서도 플레이 가능한 수준을 제공하였다.
Wind Z	-2 ~ 2	X 축과 동일한 기준으로 설정하여 풍향에 따른 편향이 한쪽으로 치우치지 않도록 하였다.

바람은 라운드(씬) 시작 시 한 번만 생성되어 게임 진행 중 변하지 않는다. 이는 동일한 라운드 내에서 사용자가 같은 환경 조건을 기준으로 여러 번의 샷을 시도하며 전략을 세울 수 있도록 한 설계이다.

4. 표면별 마찰

사용 수식

$$V_{new} = V_{old} \times \mu$$

구현 방법

FixedUpdate() 시작 시 friction 값을 매 프레임 0.99(기본값)로 초기화한다. 공이 표면 Collider 와 접촉을 유지하는 동안 OnCollisionStay()가 태그(Fairway / Rough / Sand)를 검사하여 friction 값을 해당 표면의 마찰계수로 갱신하며, 이 값은 같은 물리 스텝 내에서 수평 속도 감속 계산에 사용된다. 수평 속도 성분(x, z)에만 마찰계수를 곱해 점진적으로 감속시키고, 수직 속도(y)는 그대로 유지하여 낙하·바운스 동작과 분리하였다. 표면과 접촉하지 않는 동안(공중에 있는 동안)에는 기본값 0.99 가 적용되어 약한 공기 저항처럼 작동한다.

```
// FixedUpdate()
friction = 0.99f;
if (rb.linearVelocity.magnitude > 0.1f)
{
    Vector3 horizontalVelocity =
        new Vector3(rb.linearVelocity.x, 0, rb.linearVelocity.z);
    horizontalVelocity *= friction;
    rb.linearVelocity = new Vector3(
        horizontalVelocity.x, rb.linearVelocity.y, horizontalVelocity.z);
}

// OnCollisionStay() - 표면별 마찰계수 갱신
if (collision.gameObject.CompareTag("Fairway")) friction = 0.995f;
if (collision.gameObject.CompareTag("Rough"))   friction = 0.98f;
if (collision.gameObject.CompareTag("Sand"))     friction = 0.95f;
```

파라미터 설정 근거

표면	마찰계수 μ	설정 근거
Fairway	0.995	잔디가 짧고 매끄러운 영역이므로 가장 작은 감속을 적용하여 공이 가장 멀리 굴러가도록 하였다.
Rough	0.98	긴 잔디로 인한 저항을 반영하여 Fairway 보다 더 큰 감속을 적용하였다.
Sand	0.95	모래 벙커를 가정하여 세 표면 중 가장 큰 감속을 적용, 벙커에 빠지면 거의 구르지 못하도록 하였다.
기본값(공중)	0.99	표면과 접촉하지 않은 상태에서도 자연스러운 감속이 적용되도록 한 기본 저항값이다.

세 표면 간 마찰계수 차이는 실제 플레이 테스트에서 이동 거리 차이가 명확히 체감되는 수준으로 조정된 값이다. 다만 현재 구현에서는 friction 이 지면 접촉 여부와 무관하게 매 프레임 수평 속도에 곱해지므로, 좁은 의미의 "표면 마찰"보다는 상시 작동하는 수평 항력(드래그)에 가깝게 동작한다는 점은 4 절의 한계로 9 장에서 다시 다룬다.

5. 충돌 및 반발 (Collision & Restitution)

사용 수식

$$V_{after} = e \times V_{before}$$

여기서 e 는 반발계수(Coefficient of Restitution)이며, 본 프로젝트에서는 충돌 대상에 따라 두 가지 값을 적용하였다.

(1) 벽 충돌

Wall 태그를 가진 Collider 와 충돌하면 Vector3.Reflect 함수로 충돌 노멀에 대한 반사 벡터를 계산한 뒤, 반발계수 0.8 을 곱하여 반사 후 속도의 80%를 유지하도록 하였다.

```
// OnCollisionEnter() - 벽 반사
Vector3 reflect = Vector3.Reflect(
    rb.linearVelocity, collision.contacts[0].normal);
rb.linearVelocity = reflect * 0.8f;
```

(2) 지면 착지 바운스

공이 거의 수평인 면(충돌 노멀의 y 성분 > 0.5)에 낙하 속도($y < -1$)로 처음 닿는 순간, 수직 속도만 반전·감쇄시켜 바운스를 구현하였다. 이는 OnCollisionEnter 에서 최초 접촉 프레임에 한 번 적용되며, 착지 이후의 구름 운동은 4 장의 마찰에 의해 처리된다.

```
// OnCollisionEnter() - 지면 바운스
if (collision.contacts[0].normal.y > 0.5f)
{
    Vector3 velocity = rb.linearVelocity;
    if (velocity.y < -1f)
    {
        velocity.y = -velocity.y * 0.3f;
        rb.linearVelocity = velocity;
    }
}
```

}

파라미터 설정 근거

충돌 대상	반발계수 e	설정 근거
벽 (Wall)	0.8	벽은 비교적 단단한 재질로 가정하여 에너지 손실이 적은 반사가 일어나도록, 충돌 후 속도의 약 80%를 유지하게 하였다.
지면 (착지)	0.3	골프공이 잔디 위에서 크게 튀어 오르지 않는 실제 거동을 반영하여, 수직 속도의 30%만 남기는 낮은 값으로 설정하였다.

6. 궤적 예측 (Trajectory Prediction)

사용 수식

$$P = P_0 + Vt + \frac{1}{2}at^2 + \frac{1}{2}F_{wind}t^2$$

구현 방법

Space 키를 누르고 있는 동안(충전 중) 매 프레임 DrawTrajectory()가 호출되어, 2 번 항의 발사체 운동 수식에 바람 가속도 항을 더한 형태로 30 개의 미래 위치를 0.1 초 간격(0 ~ 2.9 초)으로 계산하고 LineRenderer 로 표 시 한 다 . currentPower, 조 준 방 향 (aimPivot.forward), 바 람 (windManager.windForce)이 모두 수식에 반영되므로 파워나 방향이 바뀌면 예측선이 실시간으로 갱신된다.

```
// DrawTrajectory()
for (int i = 0; i < points; i++)
{
    float t = i * 0.1f;
    Vector3 point =
        startPos
        + velocity * t
        + 0.5f * Physics.gravity * t * t
        + 0.5f * windManager.windForce * t * t;
    trajectoryLine.SetPosition(i, point);
}
```

점 개수(30 개)와 시간 간격(0.1 초)은 곡선을 매끄럽게 표현하면서도 매 프레임 반복 계산하는 비용을 과도하게 늘리지 않는 균형점으로 선택하였다.

7. 카메라 시스템

구현 방법

AimCamera 와 FollowCamera 두 개의 Camera GameObject 를 두고, GolfBallController 가 상황에 따라 SetActive 로 두 카메라를 전환한다. 발사(Shoot()) 시점에 AimCamera 를 비활성화하고

FollowCamera 를 활성화하며, 공의 속도가 0.1 미만으로 1 초 이상 유지되면(정지 판정) 다시 FollowCamera 를 비활성화하고 AimCamera 를 활성화한다.

```
// Shoot() - 발사 시 카메라 전환
aimCamera.gameObject.SetActive(false);
followCamera.gameObject.SetActive(true);

// FixedUpdate() - 정지 판정(1초 이상) 후 복귀
if (isBallMoving && stopTimer > 1f)
{
    isBallMoving = false;
    cameraFollow.SetAimView();
    aimCamera.gameObject.SetActive(true);
    followCamera.gameObject.SetActive(false);
}
```

FollowCamera 에는 CameraFollow 스크립트가 부착되어 있으며, LateUpdate()에서 매 프레임 공의 Rigidbody 속도 방향(정지 상태에서는 공의 정면 반대 방향)을 기준으로 목표 위치(desiredPos = 공의 위치 - 이동방향 × 12 + 위쪽 × 8)를 계산한다. 카메라 위치는 Vector3.Lerp 로, 카메라가 공을 바라보는 회전은 Quaternion.LookRotation 으로 목표 회전을 구한 뒤 Quaternion.Slerp 로 부드럽게 보간하여 급격한 시점 변화 없이 자연스럽게 추적하도록 하였다.

```
// CameraFollow.LateUpdate()
Vector3 desiredPos =
    target.position - moveDir * 12f + Vector3.up * 8f;
transform.position = Vector3.Lerp(
    transform.position, desiredPos, Time.deltaTime * 3f);

Quaternion targetRot = Quaternion.LookRotation(
    target.position - transform.position);
transform.rotation = Quaternion.Slerp(
    transform.rotation, targetRot, Time.deltaTime * 3f);
```

파라미터 설정 근거

파라미터	값	설정 근거
보간 계수	Time.deltaTime × 3	값이 너무 크면 카메라가 즉각적으로 이동해 시점이 끊기고, 너무 작으면 공을 놓치므로 부드러움과 추적성 사이의 균형으로 3 을 선택하였다.
후방 거리 / 높이	12 / 8	공 전체와 진행 방향 주변 지형이 함께 보이도록, 너무 가깝지 않으면서도 디테일을 잃지 않는 거리로 설정하였다.

8. 보조 게임 요소

핵심 물리 구현 외에 한 홀(티 박스~홀컵) 구성에 필요한 기본 요소를 다음과 같이 구현하였다.

Water Hazard

Water 태그의 Collider 와 충돌하면 스트로크를 1 증가시키고 공을 시작 위치로 되돌리며 속도를 0 으로 초기화한다.

Out of Bounds

매 FixedUpdate 마다 공의 Y 좌표가 임계값(80) 미만으로 내려가는지 검사하며, 이 경우 코스 지형 아래로 추락한 것으로 판단하여 Water Hazard 와 동일하게 스트로크 1 증가 및 시작 위치 복귀를 수행한다.

Hole In

HoleTrigger 의 OnTriggerEnter 에서 GolfBall 태그를 감지하면 공의 속도를 0 으로 만들고 "Hole In" 텍스트를 활성화하며, GolfBallController.gameClear 를 true 로 설정한다. 이후 Update()는 최상단의 gameClear 검사에 의해 모든 입력 처리를 중단하여 게임이 종료된다.

UI

StrokeUI 는 stroke 값을 텍스트로 표시하고, PowerUI 는 currentPower 값을 Slider 에 실시간으로 반영하여 사용자가 현재 충전된 파워를 시각적으로 확인할 수 있도록 하였다.

9. 한계점 및 추후 개선 방향

개발 과정에서 발견된 구현상의 한계와 개선 가능한 부분은 다음과 같다.

카메라 보간 공식 단일화

CameraFollow 에는 aimOffset, followOffset, currentOffset 필드와 SetAimView() / SetFollowView() 메서드가 선언되어 있으나, 현재 LateUpdate()의 실제 위치 계산식에서는 이 값들이 사용되지 않는다. 따라서 Aim/Follow 두 모드의 카메라 보간 공식은 사실상 동일하며, 실질적인 모드 차이는 GolfBallController 가 두 Camera GameObject 를 SetActive 로 켜고 끄는 것에 의해서만 발생한다. 추후 두 모드에 서로 다른 거리·높이 값을 적용하도록 LateUpdate 로직을 보완할 수 있다.

표면 마찰의 상시 적용

4 장에서 설명한 바와 같이 friction 은 지면 접촉 여부와 무관하게 매 FixedUpdate 마다 수평 속도에 곱해진다. 따라서 표면별 마찰이라기보다 상시 작동하는 수평 항력에 가깝게 동작한다. Raycast 나 충돌 상태 플래그로 접지 여부를 판별하여 공중에서는 마찰을 적용하지 않도록 개선할 수 있다.

바람의 고정성

바람은 씬 시작 시 한 번만 생성되어 라운드 내내 고정된다. 매 홀 또는 일정 시간 간격으로 재생성하면 더 다양한 환경을 제공할 수 있다.

Out of Bounds 판정 범위

현재 OOB 판정은 Y 좌표 임계값만을 기준으로 하여, 코스를 옆으로 크게 벗어나는 경우는 감지하지 못한다. 추후 코스 경계를 Trigger Collider 로 명시적으로 정의하면 더 정확한 판정이 가능하다.

미사용 필드 정리

GolfBallController.power 등 선언되어 있으나 현재 로직에서 사용되지 않는 필드가 일부 존재하여, 추후 코드 정리가 필요하다.

10. 결론

본 프로젝트에서는 골프 게임의 핵심 물리 요소인 발사체 운동, 바람 외력, 표면별 마찰, 충돌 및 반발, 궤적 예측, 카메라 시스템을 Unity 내장 물리 기능에 의존하지 않고 C# 스크립트 내 수식 연산으로 직접 구현하였다. 각 기능의 파라미터는 다회의 플레이 테스트를 통해 게임성과 물리적 타당성을 함께 고려하여 결정하였다.

이 과정에서 벡터의 합성과 분해, 힘과 가속도를 이용한 운동 계산, Lerp/Slerp 를 이용한 부드러운 보간 등 컴퓨터 그래픽스의 핵심 개념을 실제로 동작하는 게임에 적용해 보는 경험을 얻을 수 있었으며, 9 장에서 정리한 한계점들은 추후 개선 과제로 남긴다.